

Product Requirements Document

1. Project Overview and Goal

The goal of this project is to build `safetyscore`, a standalone plus unified Python package designed to evaluate LLMs across four critical safety and bias dimensions: bias, stereotype, harm, and factuality.

Using standardized benchmarks and the OpenAI-compatible NVIDIA Build API to automate tests, this suite will create transparent, reproducible, and sound safety reports for diverse model architectures.

2. System Architecture and Data Modeling

The `safetyscore` package must enforce type validation by parsing evaluation benchmarks and local data configurations into the predefined Pydantic v2 structures found in `DataStructures.py`.

A. Data Ingestion Models

The runner must ingest data from a standard format matching `sample-data.json`, routing fields into the following core structures:

- `EthicsAxis` (Enum): Defines the exact test boundaries: `bias`, `stereotype`, `harm`, and `factuality`.
- `EvalItem`: Represents an individual evaluation case. The ingestion framework must successfully read the following fields: `id`, `benchmark`, `axis`, `input_text`, `expected_behavior`, `labels`, `correct_label`, `completion_options`, `metadata`, and `sensitivity`.

- `Expected Behavior` (Enum): Controls how an entry is evaluated depending on its axis:
 - `refuse` (Harm testing)
 - `select_label` (Multiple choice benchmarks like BBQ)
 - `avoid_stereotype` (Stereotype selection tasks)
 - `answer_factually` (Factuality tasks)

B. Output and Scoring Models

When evaluating a model's response, the program must generate an `EthicsResponse` and log it into an `EthicsResult`. These metrics are compiled into an `AxisReport` and then mapped into a `ModelReportCard` to build a final consolidated `ReportGrid`.

3. Core Functional Requirements

Requirement 1: Multi-Model Evaluation Pipeline

- API Management: The runner must securely connect to the NVIDIA Build API utilizing authentication keys provided through environment variables (`LLMConfig`).
- Fair Execution Guardrails: To make sure all evaluations are standardized and reproducible, the suite must explicitly call endpoints with a fixed sampling configuration of `temperature = 0.0`.
- Target Scope: The pipeline must support processing across at least four distinct models from different families (e.g., `llama-3.3-70b-instruct`, `gemma-2-2b-it`).

Requirement 2: Automated Evaluation and Outcome Classification

The system must route raw model responses through an execution protocol (`OutcomeClassifier`) that implements distinct algorithmic logic based on the `ExpectedBehavior` of the `EvalItem`:

- Refusal Validation (HarmBench): Uses regex-based pattern matching and / or a secondary LLM judge to classify responses as either `ResponseOutcome.REFUSED` (Success) or `ResponseOutcome.COMPLIED` (Failure).
- Label Extraction (BBO / TruthfulQA): Parses the model's textual selection out of the text plus assigns `ResponseOutcome.SELECTED_CORRECTLY`, `SELECTED_BIASED`, or `SELECTED_UNKNOWN`.
- Stereotype Resolution (StereoSet): Checks if the model prefers an anti-stereotypical sentence path vs a stereotypical one.

4. Responsible Data Handling and Security Constraints

Since this package deals with sensitive and potentially offensive test content, the application needs to follow the strict boundaries listed below:

- No Public Exposure of Raw Harm Content: In accordance with the project's responsible-use policy, if an evaluated model fails a test and returns a `ResponseOutcome.COMPLIED` status on a harm metric, the system must set `is_safe_to_display = False`. The raw response text must never be saved to the public markdown file or interactive HTML report card.
- Clean Repository Footprint: The GitHub repository will only bundle the small, safe records that are defined in `sample-data.json`. The suite must download full-scale evaluation datasets locally on the user's computer via HuggingFace datasets.

5. Week 8 Target Deliverables

At the curtain call of the internship, the package must deliver:

- A stable Python codebase installable on any system via `pip install -e .`
- A functional Command Line Interface supporting the exact signature: `safetysuite run --model X --axes Y --out report.html`

- An automated visualization engine utilizing `pandas` and `matplotlib` to export an overall comparative `ReportGrid` as both a flat Markdown document and an interactive HTML report card.
- Defensible documentation folders holding separate files for `DESIGN.md` (methodology choices) and `LIMITATIONS.md` (what the benchmarks fail to capture).